

Practical - CO4

Rollno:2520030011

Program:

```
tejaswini@teju: ~/OS_Lab/practical$ nano virtual_address.c
GNU nano 8.7.1 virtual_address.c
#include <stdio.h>
int main(){
int x=10;
printf("Value of x = %d\n",x);
printf("Virtual address of x = %p\n",(void *)&x);
return 0;
}
```

Output:

```
tejaswini@teju:~/OS_Lab/practical$ gcc virtual_address.c -o virtual_address
tejaswini@teju:~/OS_Lab/practical$ ./virtual_address
Value of x = 10
Virtual address of x = 0x7ffc59406d74
tejaswini@teju:~/OS_Lab/practical$ |
```

Program:

```
tejaswini@teju: ~/OS_Lab/practical$ nano memory_segments.c
GNU nano 8.7.1 memory_segments.c *
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {
    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address : %p\n", (void *)main);
    printf("Data address : %p\n", (void *)&global);
    printf("BSS address : %p\n", (void *)&bss_variable);
    printf("Heap address : %p\n", (void *)heap);
    printf("Stack address : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

Output:

```
tejaswini@teju:~/OS_Lab/practical$ gcc memory_segements.c -o memory_segements
tejaswini@teju:~/OS_Lab/practical$ ./memory_segements
Code address : 0x5cea253771a9
Data address : 0x5cea2537a010
BSS address : 0x5cea2537a018
Heap address : 0x5cea48b27010
Stack address : 0x7fff887a5e8c
tejaswini@teju:~/OS_Lab/practical$
```

Program:

```
tejaswini@teju: ~/OS_Lab/practice$ nano page_table.c
GNU nano 8.7.1 page_table.c *
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};
    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

Output:

```
tejaswini@teju:~/OS_Lab/practical$ nano page_table.c
tejaswini@teju:~/OS_Lab/practical$ gcc page_table.c -o page_table
tejaswini@teju:~/OS_Lab/practical$ ./page_table
Enter page number (0-3): 1
Page Fault!
Page must be loaded into memory.
tejaswini@teju:~/OS_Lab/practical$
```

Program:

```
tejaswini@teju: ~/OS_Lab/practical$ nano address_translation.c
GNU nano 8.7.1 address_translation.c *
#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

Output:

```
tejaswini@teju:~/OS_Lab/practical$ nano address_translation.c
tejaswini@teju:~/OS_Lab/practical$ gcc address_translation.c -o address_translation
tejaswini@teju:~/OS_Lab/practical$ ./address_translation
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number      = 1
Physical Address = 150
tejaswini@teju:~/OS_Lab/practical$
```

Program:

```
tejaswini@teju: ~/OS_Lab/practical$ nano dynamic_memory_allocation.c
GNU nano 8.7.1 dynamic_memory_allocation.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}
```

Output:

```
tejaswini@teju:~/OS_Lab/practical$ gcc dynamic_memory_allocation.c -o dynamic_memory_allocation
tejaswini@teju:~/OS_Lab/practical$ ./dynamic_memory_allocation
10 20 30 40 50
tejaswini@teju:~/OS_Lab/practical$
```

Program:

```
tejaswini@teju: ~/OS_Lab/practical$ nano null_pointer.c
GNU nano 8.7.1 null_pointer.c
#include <stdio.h>

int main() {
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

Output:

```
tejaswini@teju:~/OS_Lab/practical$ nano null_pointer.c
tejaswini@teju:~/OS_Lab/practical$ gcc null_pointer.c -o null_pointer
tejaswini@teju:~/OS_Lab/practical$ ./null_pointer
Before accessing memory
Segmentation fault ./null_pointer
tejaswini@teju:~/OS_Lab/practical$ |
```